

Data Quality Audit Framework

A Structured, Repeatable Approach to Assessing & Correcting Enterprise Datasets

Author: Ratnesh Chauhan | Data Analyst & BI Developer

Version: 1.0.0

PROJECT OVERVIEW & OBJECTIVES

The **Data Quality Audit** project focuses on identifying, measuring, documenting, and resolving common data quality issues in target datasets. High-quality data is essential for reliable reporting, business intelligence, and downstream predictive analytics.

Key Objectives

- Understand data structure & schema integrity.
- Establish domain-specific validation rules.
- Quantify missing, duplicate, & out-of-bounds records.
- Clean and standardise problematic records.
- Build a repeatable audit framework.

Key Audit Checks

- Missing values & null pattern analysis.
- Duplicate record & transaction detection.
- Value range & domain boundary checks.
- Type mismatch & formatting standardization.
- Categorical label consistency.

PROJECT REPOSITORY STRUCTURE

The audit repository is structured to separate raw data, processing notebooks, output logs, and final documentation to maintain clean data lineage.

```
Data-Quality-Audit/ | ├── Data/ | ├── raw_dataset.csv # Unmodified input source data | ├── cleaned_sample.csv # Processed and validated output | ├── Notebook/ | ├── Data_Quality_Audit.ipynb # Execution logic & statistical checks | ├── Reports/ | ├── audit_report.csv # Aggregated quality scorecard | ├── issue_log.csv # Line-item defect log with severity | └── README.md # Project overview & usage instructions
```

Tools & Technologies

Tool	Role in Framework
Python	Primary programming environment for automated data processing.
Pandas	Core library for data manipulation, vectorised checks, and cleaning.
Jupyter Notebook	Interactive runtime environment for audit exploration and report building.
Matplotlib	Visual distributions and missing value pattern visualizations.



DATA QUALITY CHECKS & EXECUTION LOGIC

The audit follows a structured checklist to systematically isolate and quantify data defects across multiple dimensions using Pandas.

1. Missing Value Check

Identifies incomplete records and evaluates column completeness ratios to determine imputation or removal strategies.

```
# Identify missing counts and percentages
missing_values = df.isnull().sum()
missing_percentage = (df.isnull().mean() * 100).round(2)

missing_summary = pd.DataFrame({
    "Missing Count": missing_values,
    "Missing Percentage": missing_percentage
})
missing_summary = missing_summary[missing_summary["Missing Count"] > 0]
print(missing_summary)
```

2. Duplicate Record Check

Detects duplicate rows that inflate sales or totals.

```
duplicate_count = df.duplicated().sum()
print("Duplicate Records:", duplicate_count)
```

3. Data Type Validation

Ensures fields adhere to analytical schema.

```
print(df.dtypes)
# Verify Datetime, Numeric, String mapping
```

4. Expected Column Schema Mappings

Column Name	Expected Target Type	Validation Purpose
Order Date	Datetime	Enables time-series aggregation and date math.
Sales	Numeric (Float)	Prevents string concatenation in revenue totals.
Quantity	Integer	Ensures discrete item count metrics.
Customer ID / Category	String / Categorical	Enforces proper grouping dimensions.

5. Range Validation Logic

Checks numeric fields against real-world business constraints to highlight erroneous entries.

```
# Flag invalid negative sales transactions
invalid_sales = df[df["Sales"] < 0]
print("Invalid Sales Records:", len(invalid_sales))

# Additional Rules: Quantity > 0 | Discount [0, 1] | Rating [1, 5]
```

6. Consistency & Whitespace Checks

Removes extra spaces and standardises text capitalization to fix incorrect category split-ups.

VALIDATION RULES REGISTRY

Standardized rule registry applied across data pipelines to classify and trace data issues.

Rule ID	Validation Rule Description	Issue Type	Target Column Type
DQ001	Required fields should not be null	Missing Value	All Key Columns
DQ002	Records should be unique across natural keys	Duplicate	Primary Keys / Rows
DQ003	Sales values should not be negative	Range Error	Financial Numeric
DQ004	Quantity should be greater than zero	Range Error	Inventory Numeric
DQ005	Dates should follow valid ISO formatting	Consistency	Date/Time
DQ006	Categories must belong to approved list	Consistency	Categorical Text
DQ007	Text fields should not contain extra spaces	Formatting	All String Fields
DQ008	Numeric columns must contain numeric types	Data Type	Metrics / Measures
DQ009	IDs must follow standard regular expression	Consistency	Identifiers
DQ010	Percentage metrics must stay within [0, 100%]	Range Error	Ratios / Percentages

QUANTIFYING QUALITY & AUDIT OUTPUT

Every defect is quantified as a percentage of overall dataset volume to evaluate business risk.

Audit Executive Scorecard

Quality Check	Status
Missing Values Check	REVIEW
Duplicate Records	ACTION REQUIRED
Invalid Value Ranges	REVIEW
Data Type Mismatches	PASSED
Consistency & Formatting	REVIEW

Issue Severity Standard

HIGH: Directly corrupts key metrics (e.g. Sales, Revenue). Immediate fix needed.

MEDIUM: Affects specific sub-groupings or downstream reports. Correct before release.

LOW: Minor text formatting/spacing. Minimal impact on calculations.

Audit Issue Log Example

Issue ID	Target Field	Issue Type	Description	Severity	Action Plan
DQ001	Sales	Missing	Null value in revenue field	HIGH	Investigate / Impute
DQ002	Order ID	Duplicate	Duplicate transaction rows	HIGH	Deduplicate Records
DQ003	Quantity	Range	Negative item counts found	MEDIUM	Filter / Correct
DQ004	Category	Consistency	Inconsistent text casing	LOW	Strip & Title Case

END-TO-END AUDIT WORKFLOW

1. Load Source Dataset



2. Inspect Structure & Types



3. Apply Validation Rules



4. Run Automated Profiling



5. Quantify Issue Metrics



6. Populate Issue Log



7. Clean & Export Sample Data

QUALITY CHECKLIST

Dataset Understanding

- [✓] Row/Col counts validated
- [✓] Business meanings mapped

Completeness & Uniqueness

- [✓] Missing percentages calculated
- [✓] Primary key duplicates checked

Validity & Formatting

- [✓] Out-of-range bounds tested
- [✓] Whitespace stripped & text cased

Reporting

- [✓] Audit report generated
- [✓] Cleaned dataset exported

RECOMMENDED DATASETS FOR IMPLEMENTATION



Superstore Sales Dataset

Ideal for auditing complex multi-dimensional order data including Order IDs, Customer segments, Discounts, Sales, and Profit boundaries.



Retail Transactions Dataset

Excellent for testing duplicate transaction detection, customer ID formatting, price validation, and missing date handling.

Sample Cleaning Script Execution

```
# Create isolated copy for transformations
cleaned_df = df.copy()

# 1. Deduplicate records
cleaned_df = cleaned_df.drop_duplicates()

# 2. Trim padding white space across object columns
for col in cleaned_df.select_dtypes(include="object").columns:
    cleaned_df[col] = cleaned_df[col].str.strip()

# 3. Export cleaned sample
cleaned_df.to_csv("Data/cleaned_sample.csv", index=False)
```

KEY TECHNICAL & STRATEGIC CONCEPTS

Q1: What makes a data quality rule effective?

An effective quality rule must be **clear, measurable, business-relevant, and repeatable**. Instead of ambiguous goals like "sales figures should be correct," explicit rules like "Sales must be a positive float greater than 0" enable clear automated testing, error tracking, and actionable fixes.

Q2: How do you prioritize data quality fixes when resources are limited?

Prioritization relies on a **Business Impact vs. Effort Matrix**. Critical metrics that impact financial reporting or executive KPI dashboards (e.g. Sales, Revenue, Customer Counts) take top priority over cosmetic or minor text formatting issues.

KEY LEARNING OUTCOMES

- Mastered systematic dataset profiling using Python and Pandas.
- Developed custom validation scripts for range, type, and missingness checks.
- Constructed structured audit logs to bridge technical data fixes with business risk.
- Built repeatable data validation pipelines adaptable to varied enterprise datasets.

FUTURE FRAMEWORK ENHANCEMENTS

Automation & Monitoring

Integration with Great Expectations or Airflow pipelines for automated data quality checks and failure alerts.

Executive Dashboards

Building Power BI visual dashboards to monitor data health metrics across pipeline stages over time.

Author Profile & Contacts

Ratnesh Chauhan

Data Analyst | Business Analyst | Power BI Developer | Data Intelligence

 **GitHub:** github.com/Ratnesh8577

 **LinkedIn:** [linkedin.com/in/ratnesh-chauhan-41a113279/](https://www.linkedin.com/in/ratnesh-chauhan-41a113279/)

 **LeetCode:** leetcode.com/u/RatneshChauhan279/